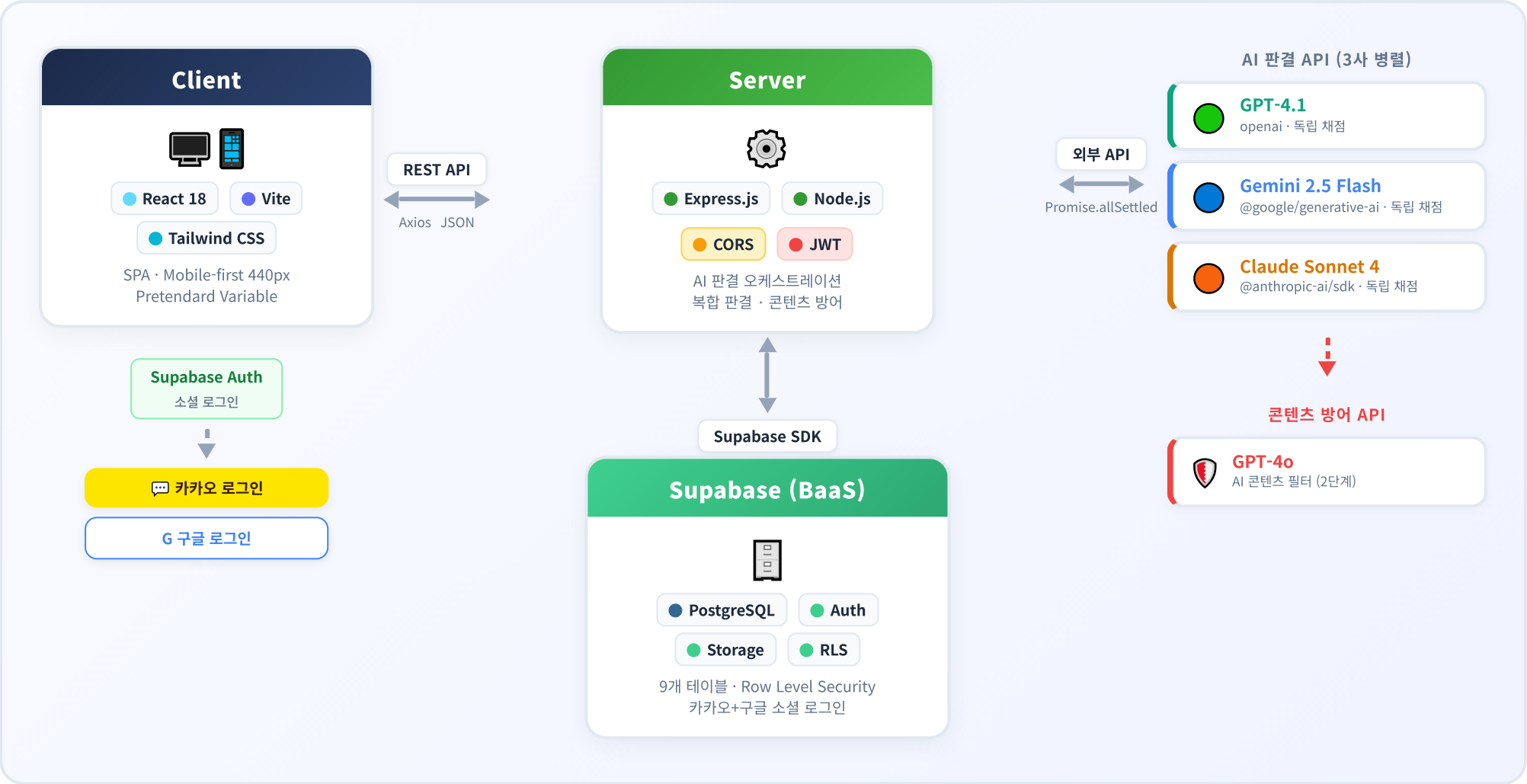
SYSTEM ARCHITECTURE

시스템 아키텍처(핵심) – Moragora

AI 논쟁 판결 플랫폼 · React + Express + Supabase



배포 : ▲ Vercel Frontend | Render Backend | Supabase DB + Auth

동기 요청/응답 · 비동기 / 간접 연결 · ◀▶ 양방향 통신

2 핵심 데이터 플로우

1. 논쟁 생성 플로우

1. Client: 3단계 위자드 (목적 → 렌즈 → 주제)
2. Axios POST /api/debates
3. Server: invite_code 생성 → Supabase INSERT
4. 초대 링크 반환 → 클립보드 복사

2. AI 판결 생성 플로우

1. 양측 주장 입력 완료 (50~2,000자)
2. Server: 3단계 콘텐츠 방어 통과 확인
3. Promise.allSettled → 3사 AI 병렬 호출
4. 5항목×20점 루브릭 채점 + 판결문 JSON
5. verdicts + ai_judgments INSERT

3. 복합 판결 (AI+시민) 플로우

1. 시민이 판결문 피드에서 A/B 투표
2. votes 테이블 INSERT (1인 1투표)
3. 30명 이상 시 citizen 점수 산정
4. 최종 = AI(60%) + 시민(40%) → verdicts UPDATE

4. 3단계 콘텐츠 방어 플로우

1. 1단계: 비속어 사전 + 개인정보 정규식
2. 2단계: GPT-4o AI 필터 (pass/warn/block)
3. 3단계: AI 게이트키핑 (주제 적합성)
4. content_filter_logs 기록

3 기술 스택 상세

구분	기술	버전 / 라이브러리	용도
Frontend	React 18	react, react-dom, react-router-dom	SPA 프레임워크
	Vite	vite 5.x	빌드 도구 (HMR)

구분	기술	버전 / 라이브러리	용도
	Tailwind CSS	tailwindcss 3.x	유틸리티 CSS (모바일 퍼스트)
Backend	Express.js	express, cors, dotenv	REST API 서버
	Node.js	v20 LTS	서버 런타임
Database	Supabase	@supabase/supabase-js	BaaS (Auth + DB + Storage)
	PostgreSQL	Supabase 내장	RDBMS (9개 테이블)
	RLS	Row Level Security	데이터 접근 제어
AI API	GPT-4.1	openai 4.x	독립 채점 및 판결
	Gemini 2.5 Flash	@google/generative-ai	독립 채점
	Claude Sonnet 4	@anthropic-ai/sdk	독립 채점 및 판결
	Grok-3-Mini	xai SDK	판결 폴백
Deploy	Vercel	vercel CLI	프론트엔드 배포 (CDN)
	Render	Docker / Node.js	백엔드 배포
	Supabase Cloud	Free tier	DB + Auth 호스팅

4 배포 구성

▲ Vercel

🚂 Render

⚡ Supabase

React + Vite 정적 빌드
CDN 글로벌 배포 · 자동 HTTPS

`moragora.vercel.app`

Express.js API 서버
Docker 컨테이너 · Auto-deploy

`moragora-api.onrender.com`

PostgreSQL + Auth + RLS
무료 티어 · 자동 백업

`xxx.supabase.co`

Moragora MVP 시스템 아키텍처 v1.0 · 2026년 3월 3일 작성

본 문서는 Moragora 프로젝트의 기술 아키텍처 문서입니다.